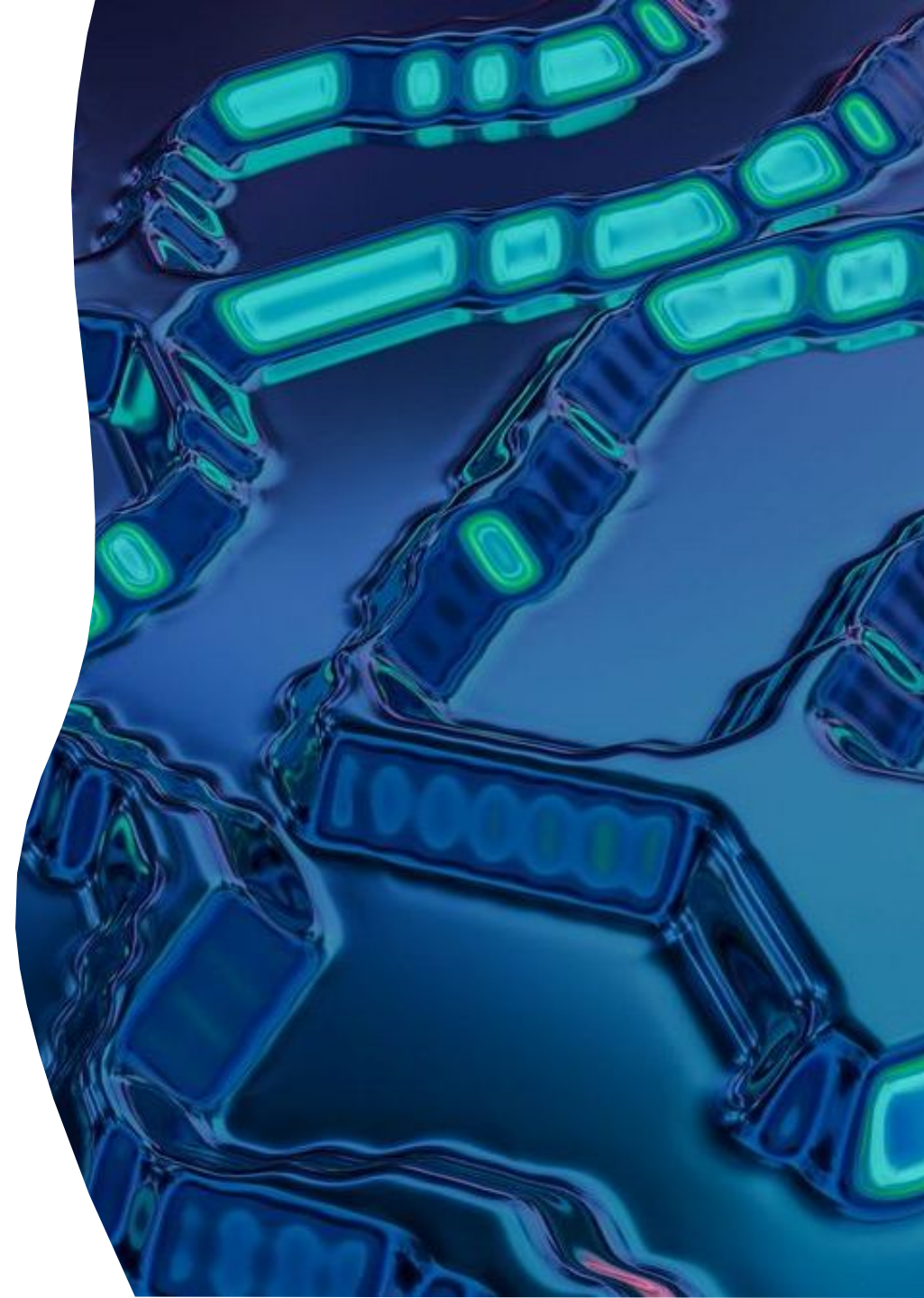


Data Analytics, Data Mining and Machine Learning

Lecture 5

Narges Chinichian · Semester 1, 2026



Overview of this course

Week	Date	Topic
1	07.09.2026	Introduction
2	14.09.2026	Data, Research Questions & Ethics
3	21.09.2026	Data Visualisation & Exploratory Data Analysis
4	28.09.2026	Preprocessing & Feature Engineering
5	05.10.2026	Machine Learning Fundamentals & Regression
6	12.10.2026	Project Presentations I
7	19.10.2026	Classification: Logistic Regression, KNN & Decision Trees
8	26.10.2026	Bank Holiday — No Class
9	02.11.2026	Gradient Descent, Regularisation & Model Complexity
10	09.11.2026	Model Evaluation & Ensemble Methods
11	16.11.2026	Neural Networks, Unsupervised Learning & Special Topics
12	23.11.2026	Project Presentations II

Quick Quiz

Check Loop



Entering the Modeling Phase

So far, we have:

01

Defined research questions and considered their ethical implications

02

Explored and visualised our data

03

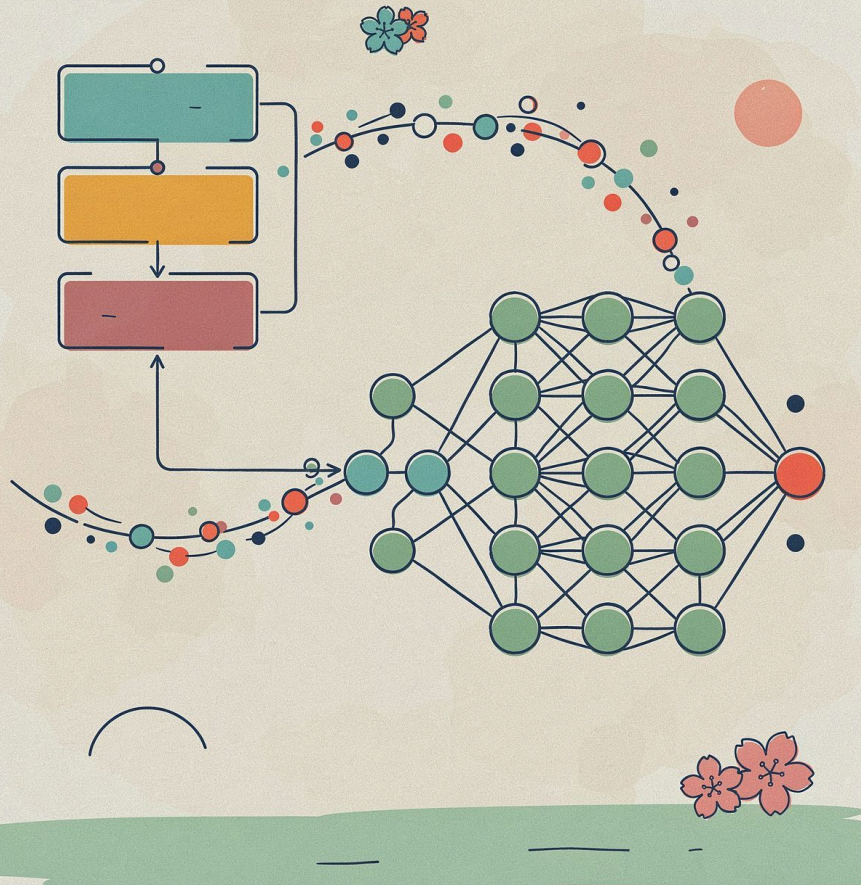
Cleaned and preprocessed our data

04

Maybe engineered some new features

Now we enter the modeling phase.

The goal is to use data to build a model that can make predictions about new observations.



The Language of Machine Learning

Dataset

The collection of observations we work with

Features (X)

The input variables used by the model

Target (y)

The variable we want to predict

Model

A function that maps features X to predictions $\hat{y}$, defined by its parameters.

Parameters

Values learned during training. Not to be confused with hyperparameters, which are set beforehand.

Prediction ($\hat{y}$)

The model's output for a given input.

Error

How far a prediction is from the observed value, measured in a way we choose.

Loss

A function that scores how wrong the predictions are. Training minimises it.

Training

Adjusting the parameters to minimise the loss on the training data.

Training Data vs. Test Data

Remember what we learned last week:

Training data

Used to learn the model's parameters.

Test data

Used to assess how the trained model performs on unseen observations.



Types of Machine Learning

Supervised Learning

The training data includes the target values (labels). The model learns to predict y from X .

Examples:

- Regression $\rightarrow$ predict a numerical value
- Classification $\rightarrow$ predict a category

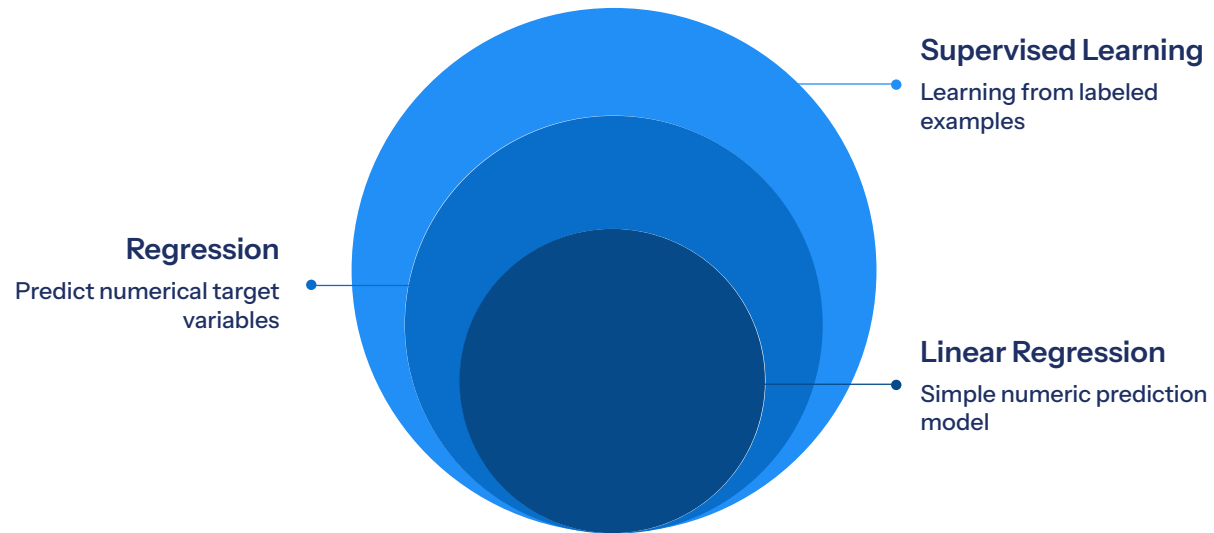
Unsupervised Learning

No labels. The model finds structure in X alone.

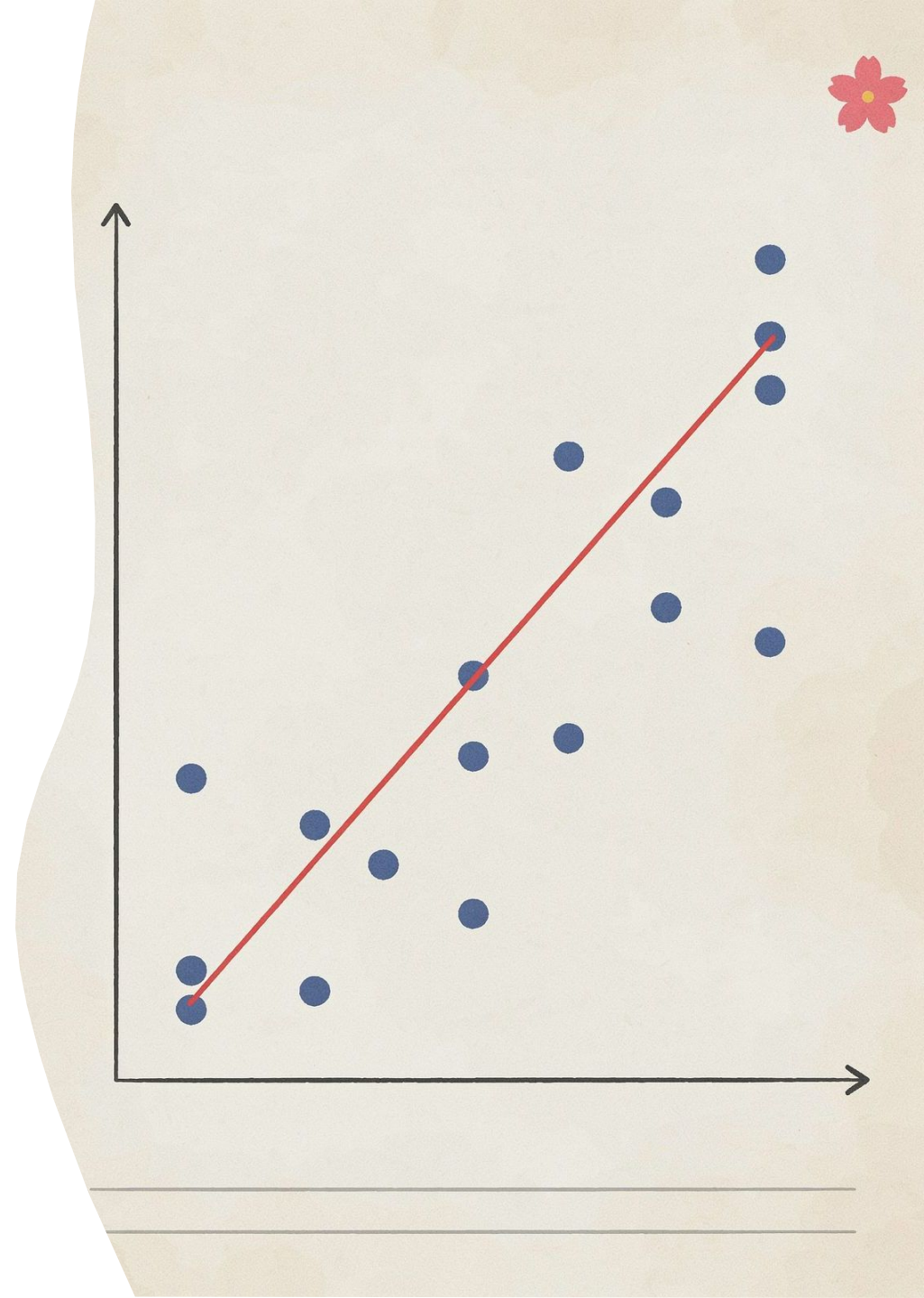
Examples:

- Clustering
- Dimensionality reduction

Where Does Linear Regression Fit?



Linear regression is one of the simplest and most useful models for predicting a **numerical target**.



What Questions Require Regression?

Regression is appropriate when our target is a **numerical quantity**.

Examples:

- How much will a house cost?
- How much energy will a building consume?
- How long will a delivery take?
- What will a student's exam score be?

The common question:

Can we predict a numerical value from one or more features?

Linear Regression

Remember the Equation of a Line?

You may remember:

$$y = mx + b$$

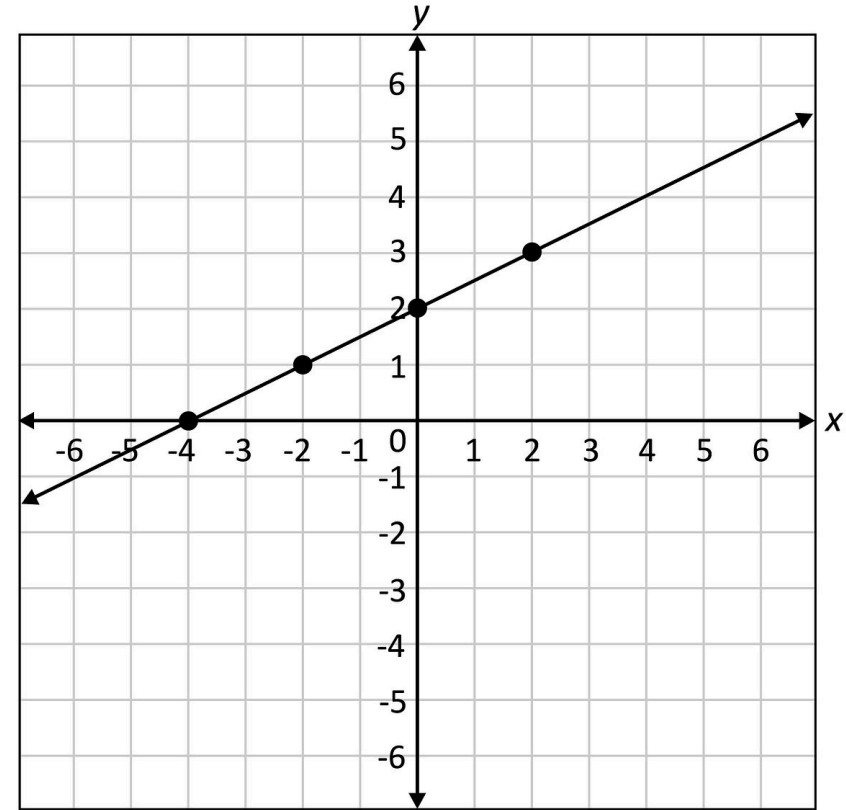
A straight line is defined by:

Slope (m)

Intercept (b)

Linear regression uses this familiar idea to build a predictive model.

The image shows $y = \frac{1}{2}x + 2$



From a Line to a Model

For a simple linear regression:

$$\hat{y} = b_0 + b_1 x$$

Where:

(x)

Input feature

$(\hat{y})$

Predicted target

(b_0)

Intercept

(b_1)

Coefficient / slope

The model uses (x) to predict (y) .

Example: House Prices

Suppose we want to predict house prices from several features.

Features:

Floor area in m², #Bedrooms, distance from city centre, building age

Target:

House price in €

We collect observations:

Features → House price

We have a [Multiple] Linear Regression:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \cdots + b_px_p$$

Amount	Description
120,000 €	base price
+ 3,800 €	for each m ² of floor area
+ 15,000 €	for each bedroom
- 9,500 €	for each km from the city centre
- 1,200 €	for each year of building age
= $\hat{y}$ €	estimated house price

Interpreting Regression Coefficients

Suppose our model contains:

$$\hat{y} = 120,000 + 3,800x_1 + \dots$$

where (x_1) is floor area in m².

The coefficient **3,800** means:

- According to this model, an increase of one square metre is associated with an increase of €3,800 in the predicted house price, holding the other included features constant.

Important:

A regression coefficient describes a relationship in the model. It does not automatically establish causation.

Predictions and Residuals

For every observation, the model produces a prediction.

$$\text{Residual} = y - \hat{y}$$

The residual is the difference between:

Observed value

Predicted value

A good model should produce predictions that are generally close to the observed values.

Loss: Scoring a Model

Each observation has a residual. But to compare models, we need one number for the whole dataset.

A **loss function** combines the individual errors into a single score: lower = better fit.

The data is fixed; the parameters change → the loss is a function of the parameters.

How we combine the errors is a choice, and that choice defines what "best" means:

Absolute errors

Every euro of error counts equally.

Squared errors

Large errors count disproportionately more.

📌 **Training = finding the parameters with the lowest loss.**

Least Squares

One way to measure the total prediction error is to calculate the **sum of squared residuals**:

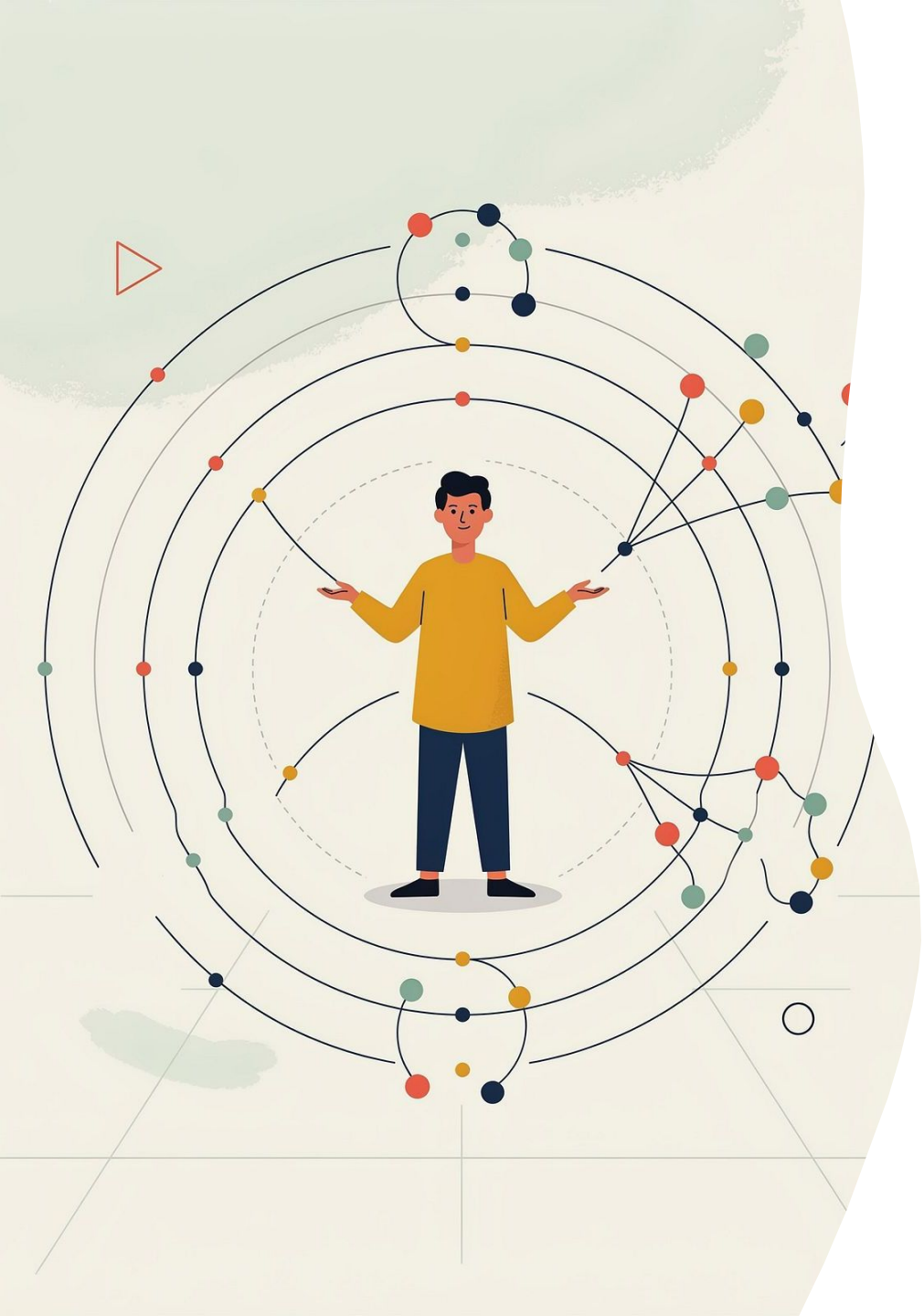
$$\sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Linear regression using ordinary least squares chooses the parameters that minimise this quantity.

Parameters → Predictions → Residuals → Squared → Summed = Loss (one particular type of loss)

different parameters → different loss values

Training goal: find the parameters that minimise this loss on the training data. For linear regression, this is called *ordinary least squares* (OLS).



Generalisation

The real goal of machine learning is **not** simply to fit the training data.

The goal is to make useful predictions for **new, unseen observations**.



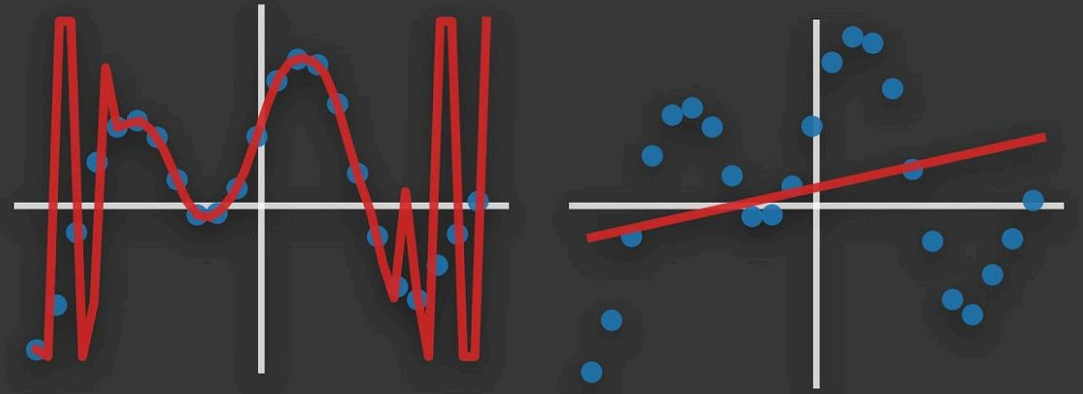
Generalisation

The ability of a model to perform well on data that it has not seen during training.

This is why we separate training and test data.

Underfitting and Overfitting

Overfitting vs **Underfitting**



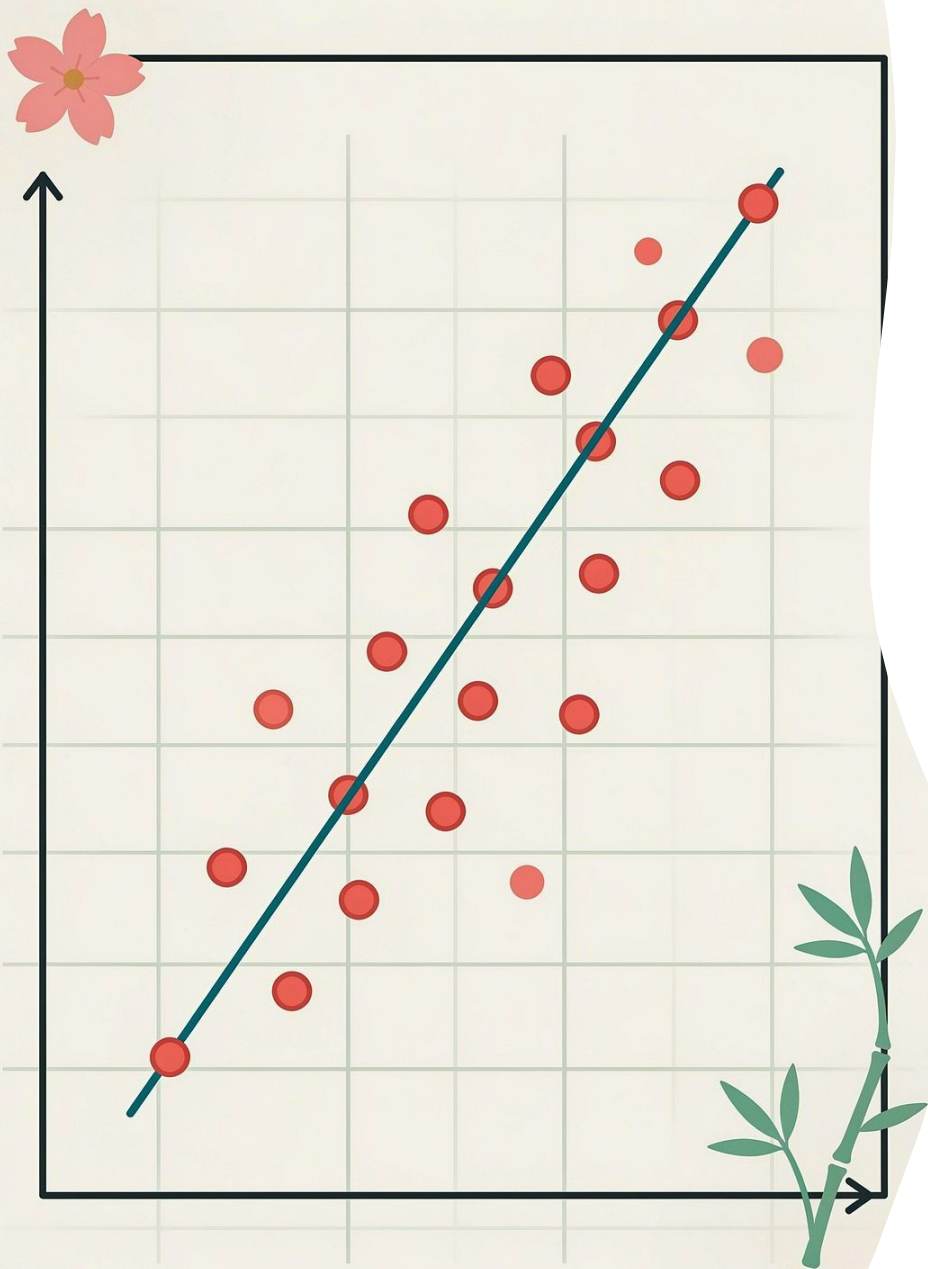
Underfitting

The model is too simple to capture important patterns in the data.

Poor training performance



Poor performance on unseen data too.



Good Fit

The model captures meaningful patterns while remaining able to generalise to new observations.

Good training performance



Good performance on unseen data

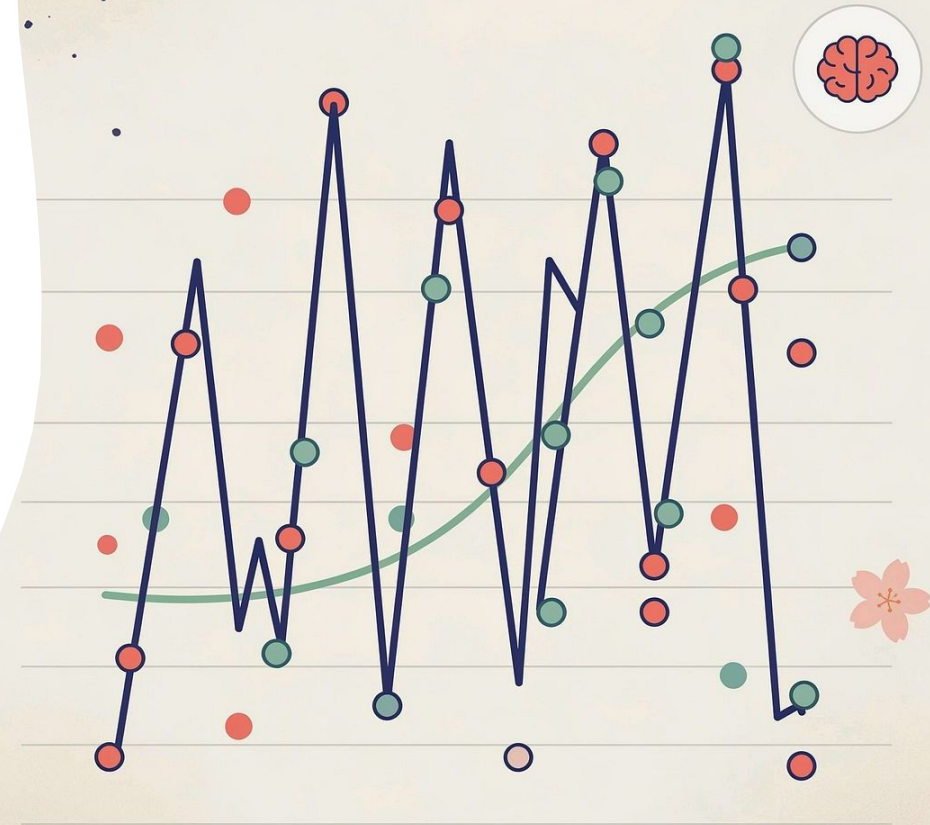
Overfitting

The model captures details and noise that are specific to the training data.

Very good training performance



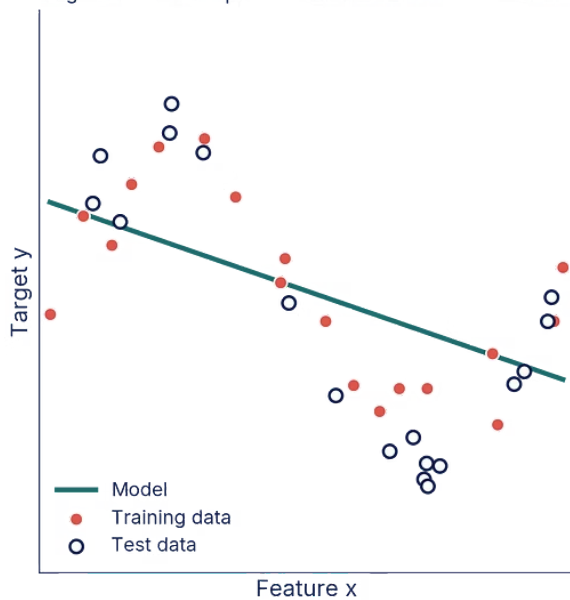
Poor performance on unseen data



A visual example

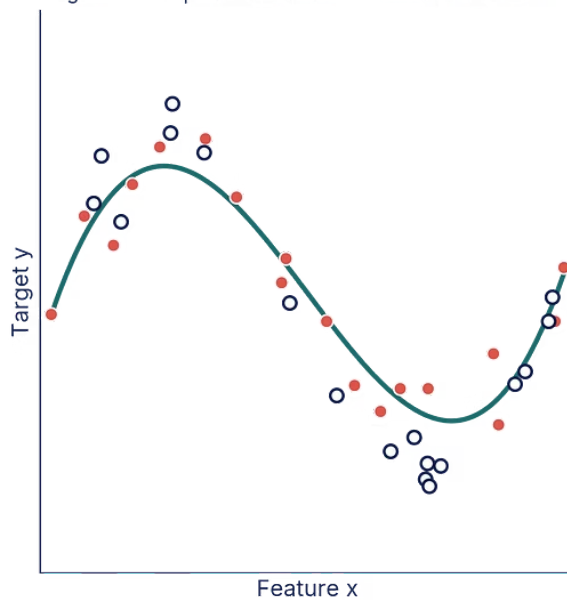
Underfitting

Degree 1 — too simple · train RMSE 0.47 · test RMSE 0.66



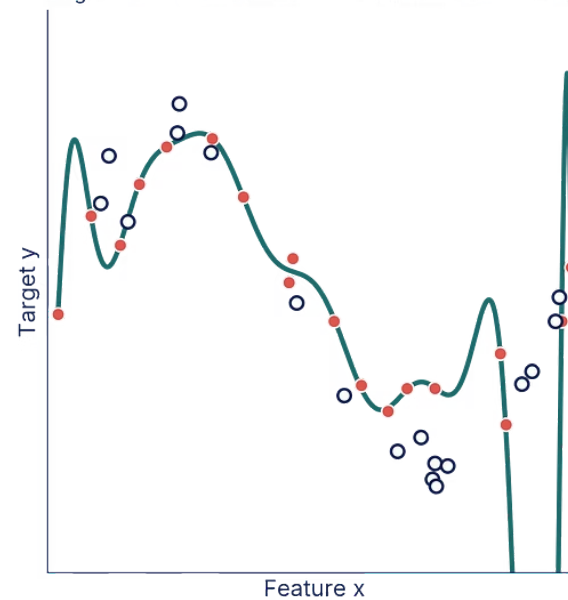
Good fit

Degree 3 — captures the trend · train RMSE 0.18 · test RMSE 0.29



Overfitting

Degree 15 — fits the noise · train RMSE 0.03 · test RMSE 1.79



Model Complexity

There is a trade-off:



We will return to **model complexity and regularisation** later in the course.

How Do We Evaluate a Regression Model?

We need to quantify how close our predictions are to the observed values on test data.



Evaluation Metrics

There are many metrics; we focus on four:

1

MAE

Mean Absolute Error

Average size of the errors

2

MSE

Mean Squared Error

Average squared error, which punishes large errors

3

RMSE

Root Mean Squared Error

Like MSE, but in the target's units

4

R^2

Coefficient of Determination

How much better we are than predicting the mean

Mean Absolute Error — MAE

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Interpretation:

On average, how far are our predictions from the actual values?

Mean Squared Error — MSE

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Because errors are squared, large errors receive more weight.

Root Mean Squared Error — RMSE

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the **same units as the target variable**.

For example:

If we predict house prices in €, RMSE is also measured in €.

R^2 — Coefficient of Determination

(R^2) describes how much of the variation in the target is explained by the model relative to a baseline model.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Where:

SS_{res}

The squared error of our model: $\sum (y_i - \hat{y}_i)^2$

SS_{tot}

The squared error of the baseline model, which always predicts the mean $\bar{y}$: $\sum (y_i - \bar{y})^2$

But:

A high R^2 does not automatically mean that a model is useful, accurate for every observation, or causal.

$R^2 = 1$

Perfect predictions

$R^2 = 0$

No better than predicting the mean

$R^2 < 0$

Worse than the mean (can happen on test data)

Putting It All Together

We can now describe the basic supervised learning process:

01

Define the problem

What do we want to predict?

02

Define (X) and (y)

Which features will we use?

03

Split the data

Training data + Test data

04

Train the model

Learn the model parameters

05

Make predictions

Generate ($\hat{y}$)

06

Calculate errors

Compare (y) and ($\hat{y}$)

07

Evaluate generalisation

How well does the model perform on unseen data?

Hands-On: Build a Regression Model

We will use Python and scikit-learn to:

Step	In Python
Define X and y	<pre>X = df[...], y = df["price"]</pre>
Split the data	<pre>train_test_split(X, y, test_size=0.2)</pre>
Train the model	<pre>model = LinearRegression() model.fit(X_train, y_train)</pre>
Inspect parameters	<pre>model.intercept_, model.coef_</pre>
Predict	<pre>y_pred = model.predict(X_test)</pre>
Evaluate	<pre>mean_absolute_error(y_test, y_pred), r2_score(...)</pre>

Key Takeaways

By the end of this session, you should be able to:

- Explain the basic terminology of machine learning
- Distinguish supervised and unsupervised learning
- Explain what makes a problem a regression problem
- Describe how linear regression works
- Interpret coefficients and predictions
- Explain residuals and least squares
- Describe underfitting and overfitting
- Calculate and interpret basic regression evaluation metrics
- Train a simple regression model using Python and scikit-learn